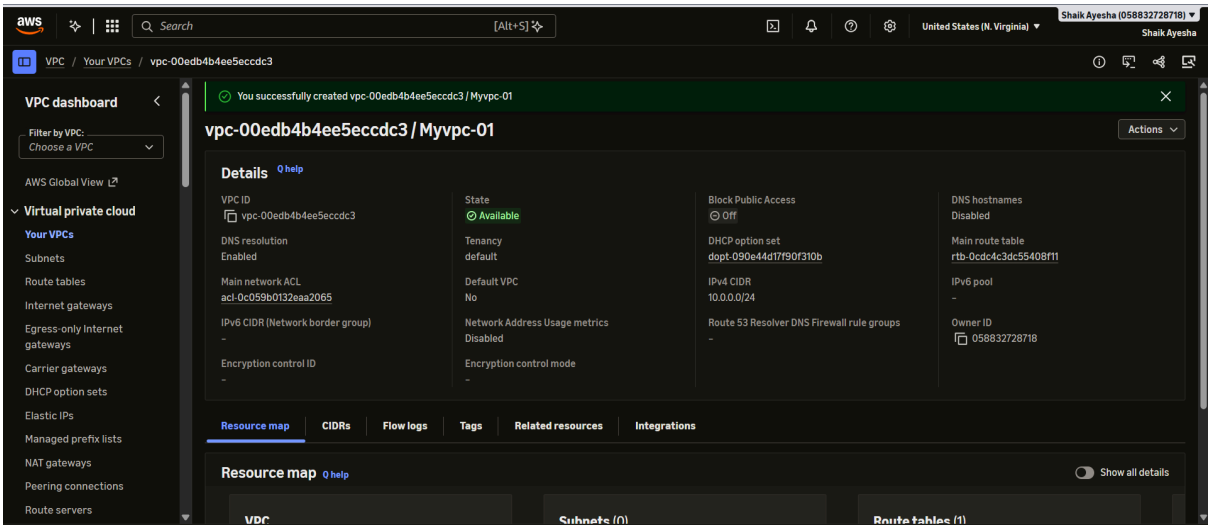
1. Create one VPC in N. Virginia region

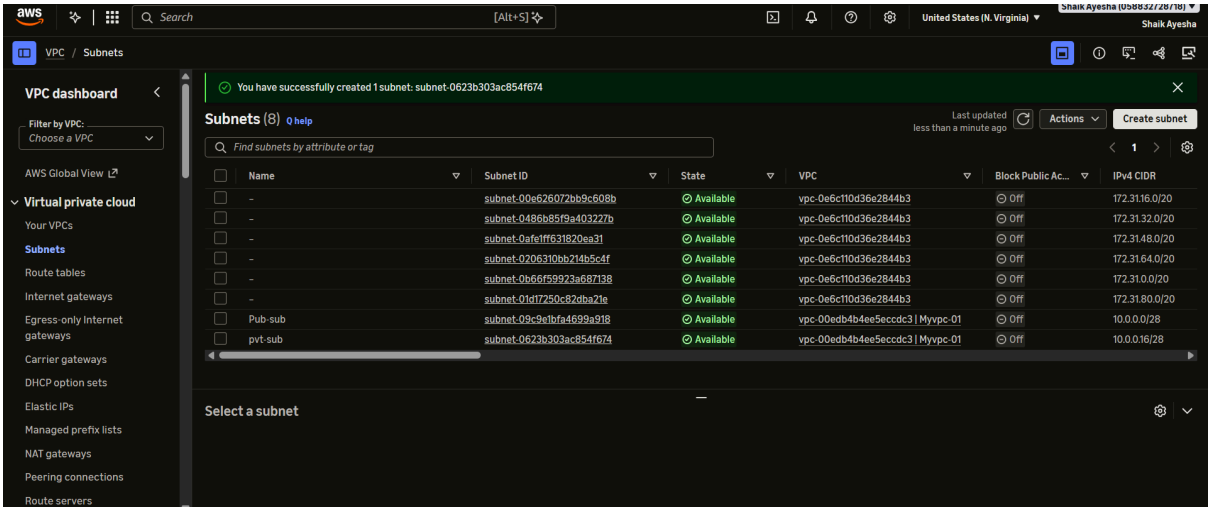


1. I logged in to the AWS Management Console.
2. I searched for VPC in the AWS search bar.
3. I opened the VPC Dashboard.
4. I selected the N. Virginia (us-east-1) region from the top-right corner.
5. I clicked on Your VPCs from the left-side navigation panel.
6. I clicked on Create VPC.
7. I selected VPC only.
8. I entered the following details:

- Name tag: My-VPC
- IPv4 CIDR block: 10.0.0.0/24
- IPv6 CIDR block: No IPv6 CIDR block
- Tenancy: Default

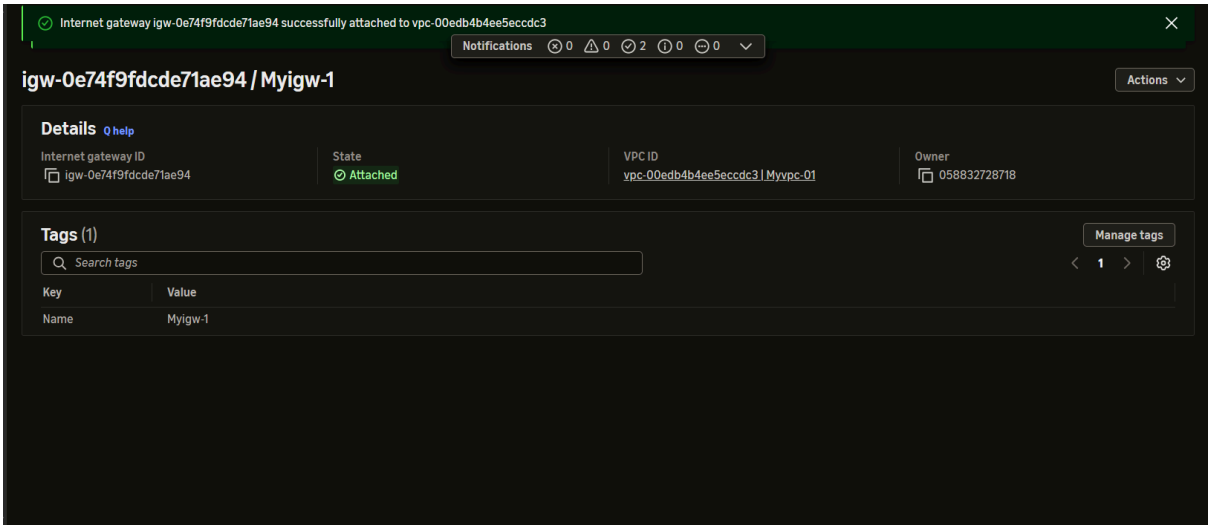
9. I selected IPv4 CIDR manual input.
10. I kept the remaining settings as default.
11. I clicked on Create VPC.
12. I verified that the VPC was successfully created under Your VPCs.
13. I noted the generated VPC ID.

2. Create two subnets: one public subnet and one private subnet.



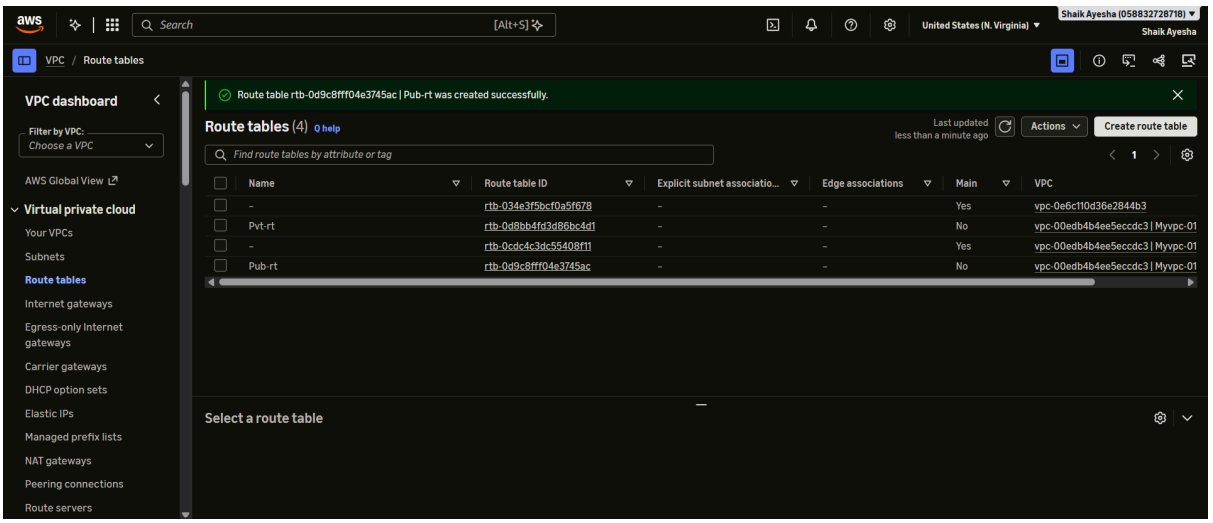
1. I logged in to the AWS Management Console.
2. I opened the VPC Dashboard.
3. I selected the N. Virginia (us-east-1) region.
4. I clicked on Subnets from the left-side navigation panel.
5. I clicked on Create subnet.
6. I selected my existing VPC (My-VPC).
7. I created the public subnet
8. I clicked on Add new subnet.
9. I created the private subnet
10. I clicked on Create subnet.
11. I verified that both subnets were successfully created under Subnets.
12. I selected Public-Subnet, clicked on Actions, and enabled Edit subnet settings → Enable auto-assign public IPv4 address.
13. I saved the changes.
14. I verified that the public subnet and private subnet were available in my VPC.

3. Attach an IGW to the VPC



1. I logged in to the AWS Management Console.
2. I opened the VPC Dashboard.
3. I selected the N. Virginia (us-east-1) region.
4. I clicked on Internet Gateways from the left-side navigation panel.
5. I clicked on Create internet gateway.
6. I entered the following details:
    - Name tag: My-IGW
7. I clicked on Create internet gateway.
8. I selected the newly created Internet Gateway (My-IGW).
9. I clicked on Actions.
10. I selected Attach to a VPC.
11. I selected my VPC (My-VPC).
12. I clicked on Attach internet gateway.
13. I verified that the IGW status changed to Attached.
14. I confirmed that the Internet Gateway was successfully attached to my VPC.

4. Create one public route table (RT) and one private route table.



1. I logged in to the AWS Management Console.
2. I opened the VPC Dashboard.
3. I selected the N. Virginia (us-east-1) region.
4. I clicked on Route Tables from the left-side navigation panel.
5. I clicked on Create route table.

1. Create Public Route Table

6. I entered the following details:
    - Name: Public-RT
    - VPC: My-VPC
7. I clicked on Create route table.
8. I selected Public-RT.
9. I opened the Routes tab.
10. I clicked on Edit routes.
11. I clicked on Add route.
12. I entered the following details:
    - Destination: 0.0.0.0/0
    - Target: Internet Gateway (My-IGW)
13. I clicked on Save changes.
14. I opened the Subnet associations tab.
15. I clicked on Edit subnet associations.
16. I selected Public-Subnet.
17. I clicked on Save associations.

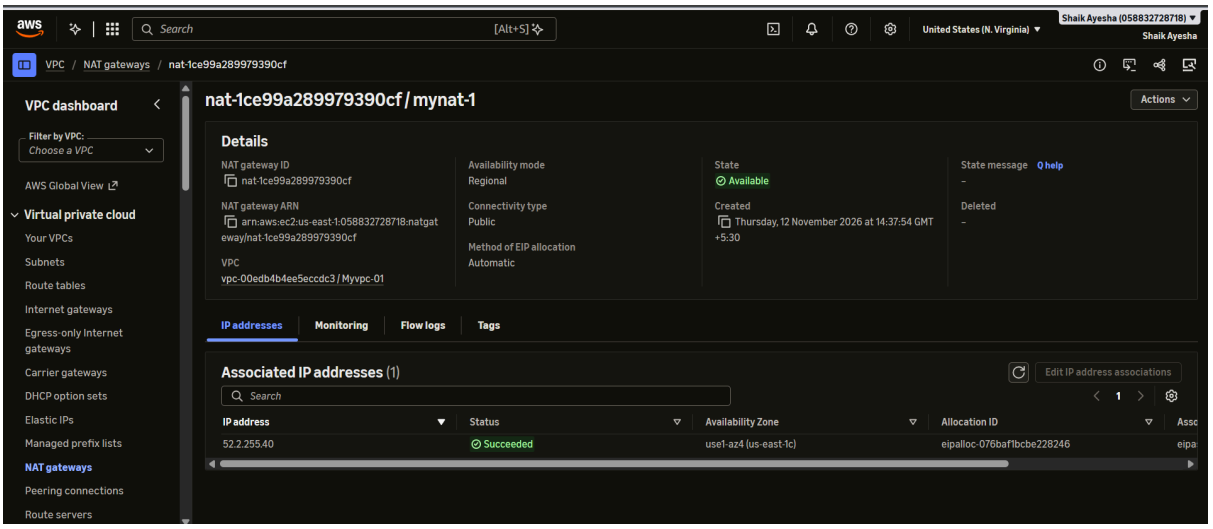
2. Create Private Route Table

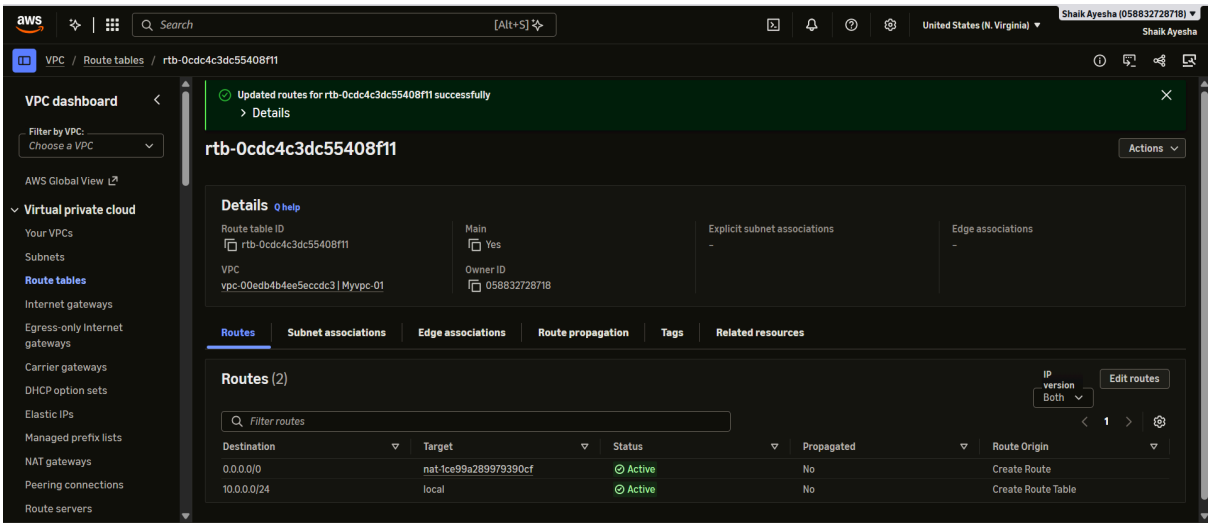
18. I clicked on Create route table.
19. I entered the following details:

- Name: Private-RT
- VPC: My-VPC

20. I clicked on Create route table.
21. I selected Private-RT.
22. I opened the Subnet associations tab.
23. I clicked on Edit subnet associations.
24. I selected Private-Subnet.
25. I clicked on Save associations.
26. I verified that the public and private subnets were associated with their respective route tables.

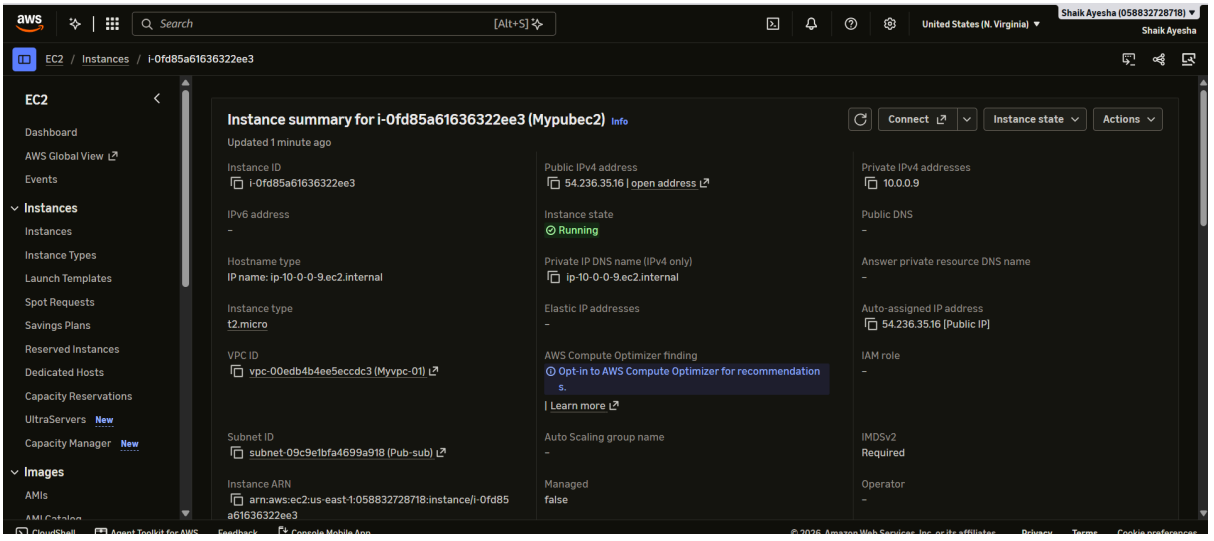
5. Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.

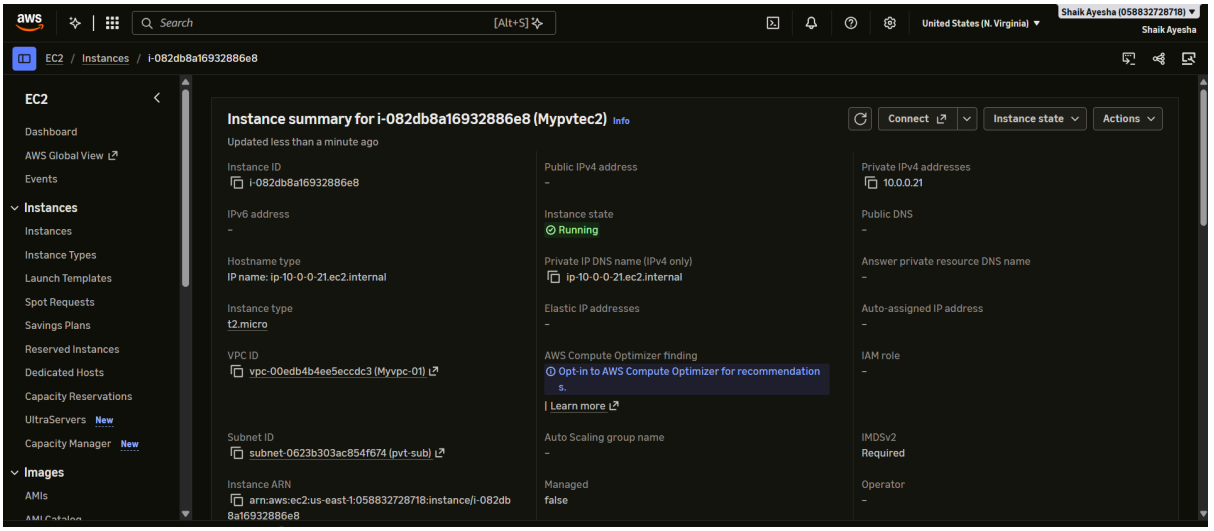




1. I logged in to the AWS Management Console.
2. I opened the VPC Dashboard.
3. I selected the N. Virginia (us-east-1) region.
4. I clicked on NAT Gateways from the left-side navigation panel.
5. I clicked on Create NAT gateway.
6. I entered the following details:
    - Name: My-NAT-Gateway
    - Subnet: Public-Subnet
    - Connectivity type: Public
7. I selected Allocate Elastic IP.
8. I clicked on Create NAT gateway.
9. I waited until the NAT Gateway status changed to Available.
10. I clicked on Route Tables from the left-side navigation panel.
11. I selected Private-RT.
12. I opened the Routes tab.
13. I clicked on Edit routes.
14. I clicked on Add route.
15. I entered the following details:
    - Destination: 0.0.0.0/0
    - Target: NAT Gateway (My-NAT-Gateway)
16. I clicked on Save changes.
17. I verified that the private route table was updated with the NAT Gateway route.
18. I confirmed that Private-Subnet was associated with Private-RT.

6. Create two instances, one in the public subnet and one in the private subnet.





1. Create a Public EC2 Instance

1. I logged in to the AWS Management Console.
2. I opened the EC2 Dashboard.
3. I selected the N. Virginia (us-east-1) region.
4. I clicked on Instances from the left-side navigation panel.
5. I clicked on Launch instances.
6. I entered the following details:
    - Name: Public-EC2
    - AMI: Amazon Linux 2023
    - Instance type: t2.micro
7. I selected an existing key pair or created a new key pair.
8. Under Network settings, I clicked on Edit.
9. I selected the following:
    - VPC: My-VPC
    - Subnet: Public-Subnet
    - Auto-assign public IP: Enable
10. I created a security group named Public-SG.
11. I added the following inbound rules:
    - SSH – TCP – Port 22 – My IP
12. I kept the remaining settings as default.
13. I clicked on Launch instance.
14. I verified that Public-EC2 was running.

2. Create a Private EC2 Instance

15. I clicked on Launch instances.

16. I entered the following details:
    - Name: Private-EC2
    - AMI: Amazon Linux 2023
    - Instance type: t2.micro
17. I selected the same key pair used for the public EC2 instance.
18. Under Network settings, I clicked on Edit.
19. I selected the following:
    - VPC: My-VPC
    - Subnet: Private-Subnet
    - Auto-assign public IP: Disable
20. I created a security group named Private-SG.
21. I added the following inbound rule:
    - SSH – TCP – Port 22 – Public EC2 private IP (/32)
22. I kept the remaining settings as default.
23. I clicked on Launch instance.
24. I verified that Private-EC2 was running.

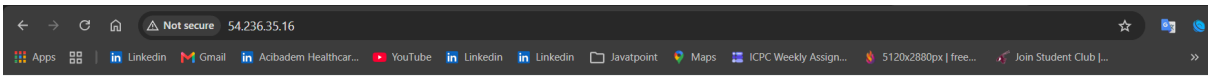
7. Deploy Apache server on both EC2 instances with a sample index.html file.

```
ec2-user@ip-10-0-0-9:~
  Verifying        : apr-util-1.6.5-1.amzn2023.0.1.x86_64                                  2/13
  Verifying        : apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64                             3/13
  Verifying        : apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64                          4/13
  Verifying        : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch                     5/13
  Verifying        : httpd-2.4.69-1.amzn2023.0.2.x86_64                                    6/13
  Verifying        : httpd-core-2.4.69-1.amzn2023.0.2.x86_64                               7/13
  Verifying        : httpd-filesystem-2.4.69-1.amzn2023.0.2.noarch                         8/13
  Verifying        : httpd-tools-2.4.69-1.amzn2023.0.2.x86_64                              9/13
  Verifying        : libbrotli-1.1.0-1.amzn2023.0.2.x86_64                                10/13
  Verifying        : mailcap-2.1.49-3.amzn2023.0.3.noarch                                 11/13
  Verifying        : mod_http2-1:2.4.69-1.amzn2023.0.2.x86_64                             12/13
  Verifying        : mod_lua-2.4.69-1.amzn2023.0.2.x86_64                                 13/13

Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                    apr-util-1.6.5-1.amzn2023.0.1.x86_64
  apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64          apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64
  generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch  httpd-2.4.69-1.amzn2023.0.2.x86_64
  httpd-core-2.4.69-1.amzn2023.0.2.x86_64            httpd-filesystem-2.4.69-1.amzn2023.0.2.noarch
  httpd-tools-2.4.69-1.amzn2023.0.2.x86_64           libbrotli-1.1.0-1.amzn2023.0.2.x86_64
  mailcap-2.1.49-3.amzn2023.0.3.noarch               mod_http2-1:2.4.69-1.amzn2023.0.2.x86_64
  mod_lua-2.4.69-1.amzn2023.0.2.x86_64

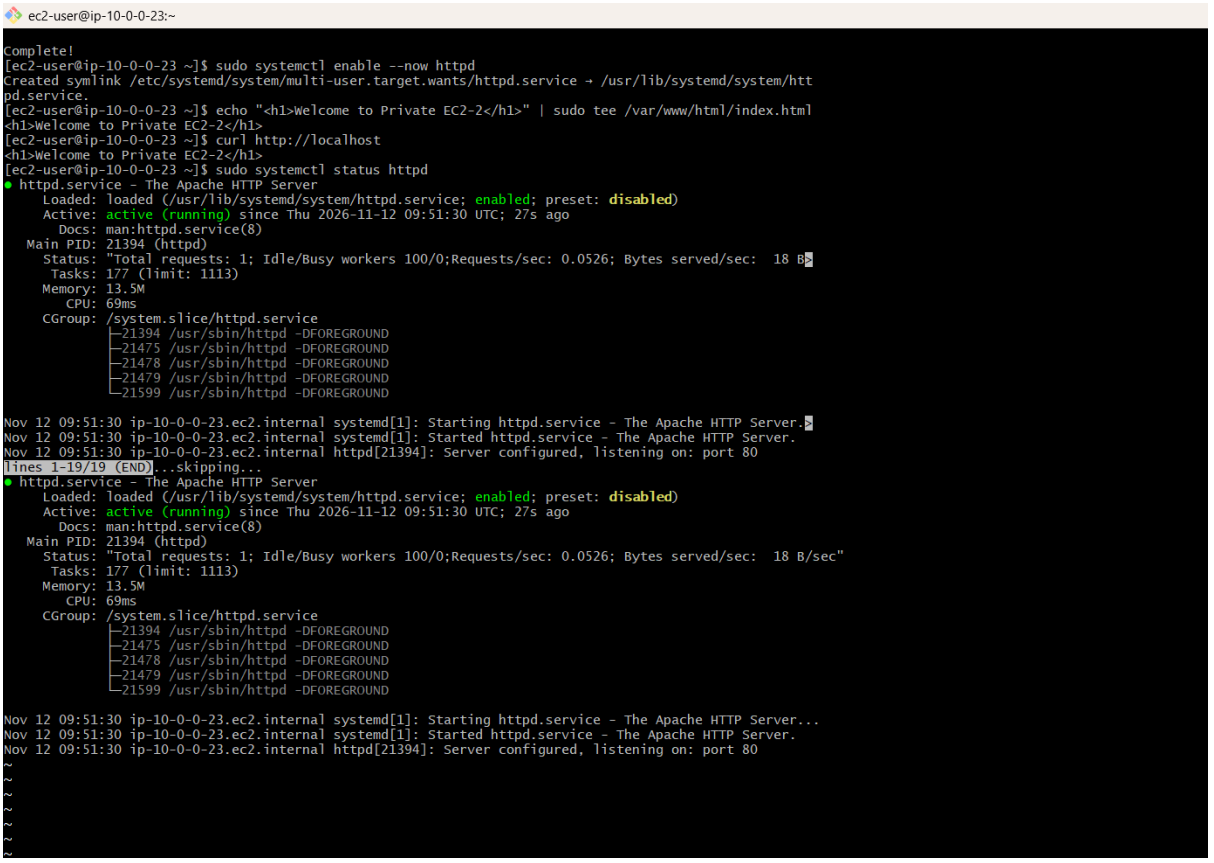
Complete!
[ec2-user@ip-10-0-0-9 ~]$ sudo systemctl start httpd
[ec2-user@ip-10-0-0-9 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
     Active: active (running) since Thu 2026-11-12 09:22:14 UTC; 6s ago
       Docs: man:httpd.service(8)
   Main PID: 12892 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 177 (limit: 1113)
     Memory: 13.3M
        CPU: 52ms
     CGroup: /system.slice/httpd.service
             ├─12892 /usr/sbin/httpd -DFOREGROUND
             ├─12983 /usr/sbin/httpd -DFOREGROUND
             ├─12988 /usr/sbin/httpd -DFOREGROUND
             ├─12989 /usr/sbin/httpd -DFOREGROUND
             └─13015 /usr/sbin/httpd -DFOREGROUND

Nov 12 09:22:14 ip-10-0-0-9.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Nov 12 09:22:14 ip-10-0-0-9.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Nov 12 09:22:14 ip-10-0-0-9.ec2.internal httpd[12892]: Server configured, listening on: port 80
[ec2-user@ip-10-0-0-9 ~]$ sudo vi /var/www/html/index.html
[ec2-user@ip-10-0-0-9 ~]$ sudo systemctl restart httpd
[ec2-user@ip-10-0-0-9 ~]$ curl http://localhost
<!DOCTYPE html>
<html>
<head>
    <title>Public Server</title>
</head>
<body>
    <h1>Welcome to Public EC2 Server</h1>
    <p>Apache is running successfully on Public EC2.</p>
</body>
</html>
[ec2-user@ip-10-0-0-9 ~]$
```



Welcome to Public EC2 Server

Apache is running successfully on Public EC2.



1. Deploy Apache on Public-EC2

1. I connected to Public-EC2 using SSH.
2. I updated the packages.

```
None
        sudo dnf update -y
```

3. I installed Apache.

```
None
        sudo dnf install httpd -y
```

4. I started and enabled Apache.

```
None
        sudo systemctl start httpd
        sudo systemctl enable httpd
```

5. I created the sample HTML file.

```
None
        sudo vi /var/www/html/index.html
```

6. I added the following content:

```
None
        <h1>Welcome to Public EC2 Server</h1>
        <p>Apache is running successfully on Public EC2.</p>
```

7. I saved the file and restarted Apache.

```
None
        sudo systemctl restart httpd
```

2. Deploy Apache on Private-EC2

8. I connected to Private-EC2 through Public-EC2 using SSH.
9. I updated the packages.

```
None
        sudo dnf update -y
```

10. I installed Apache.

```
None
        sudo dnf install httpd -y
```

11. I started and enabled Apache.

```
None
        sudo systemctl start httpd
        sudo systemctl enable httpd
```

12. I created the sample HTML file.

```
None
sudo vi /var/www/html/index.html
```

13. I added the following content:

```
None
<h1>Welcome to Private EC2 Server</h1>
<p>Apache is running successfully on Private EC2.</p>
```

14. I saved the file and restarted Apache.

```
None
sudo systemctl restart httpd
```

3. Verify both Apache servers

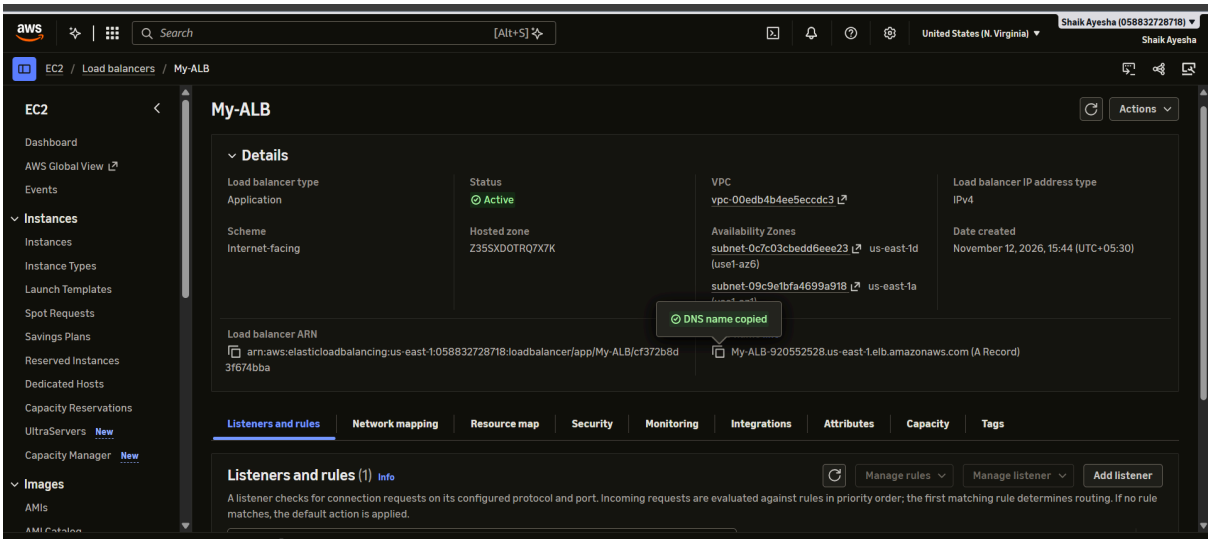
15. I checked the Apache service status on both instances.

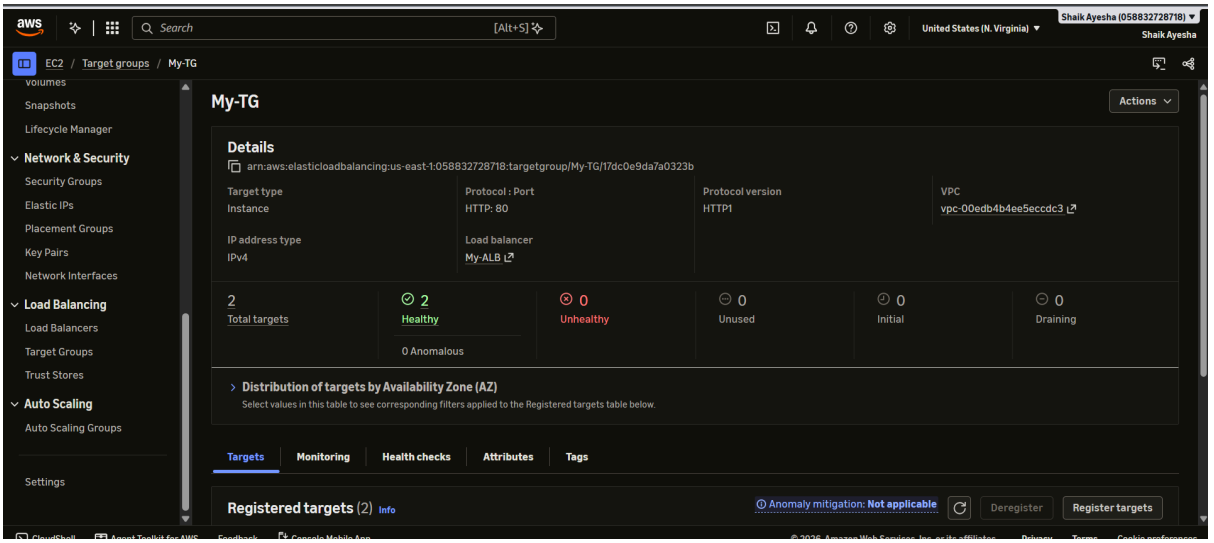
```
None
sudo systemctl status httpd
```

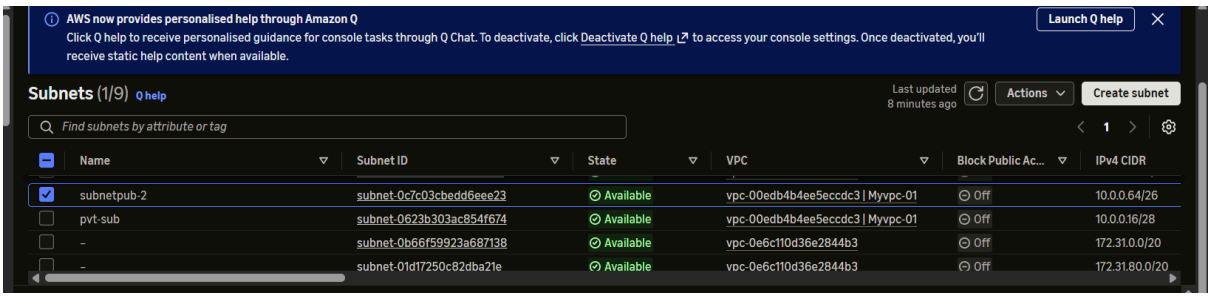
16. I verified the public webpage using the Public-EC2 public IP.
17. I verified the private webpage from Public-EC2 using the Private-EC2 private IP.

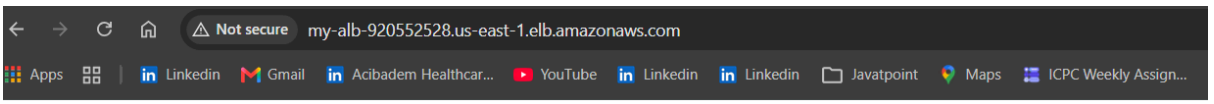
```
None
curl http://PRIVATE-IP
```

8. Create one application load balancer and attach it to both EC2 instances.


aws
Search
[Alt+S]
United States (N. Virginia)
Shaik Ayesha (058832728718)
Shaik Ayesha
EC2 / Load balancers / My-ALB
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
My-ALB
Actions
Details
Load balancer type
Application
Status
Active
VPC
vpc-00edb4b4ee5eccdc3
Load balancer IP address type
IPv4
Scheme
Internet-facing
Hosted zone
Z35SXDOTRQ7X7K
Availability Zones
subnet-0c7c03cbedd6eee23 us-east-1d (use1-az6)
subnet-09c9e1bfa4699a918 us-east-1a
Date created
November 12, 2026, 15:44 (UTC+05:30)
DNS name copied
Load balancer ARN
arn:aws:elasticloadbalancing:us-east-1:058832728718:loadbalancer/app/My-ALB/cf372b8d3f674bba
My-ALB-920552528.us-east-1.elb.amazonaws.com (A Record)
Listeners and rules
Network mapping
Resource map
Security
Monitoring
Integrations
Attributes
Capacity
Tags
Listeners and rules (1) Info
Manage rules
Manage listener
Add listener
A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.


aws
Search
[Alt+S]
United States (N. Virginia)
Shaik Ayesha (058832728718)
Shaik Ayesha
EC2 / Target groups / My-TG
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Settings
My-TG
Actions
Details
arn:aws:elasticloadbalancing:us-east-1:058832728718:targetgroup/My-TG/17dc0e9da7a0323b
Target type
Instance
Protocol : Port
HTTP: 80
Protocol version
HTTP1
VPC
vpc-00edb4b4ee5eccdc3
IP address type
IPv4
Load balancer
My-ALB
2
Total targets
2
Healthy
0 Anomalous
0
Unhealthy
0
Unused
0
Initial
0
Draining
Distribution of targets by Availability Zone (AZ)
Select values in this table to see corresponding filters applied to the Registered targets table below.
Targets
Monitoring
Health checks
Attributes
Tags
Registered targets (2) Info
Anomaly mitigation: Not applicable
Deregister
Register targets


AWS now provides personalised help through Amazon Q
Click Q help to receive personalised guidance for console tasks through Q Chat. To deactivate, click Deactivate Q help to access your console settings. Once deactivated, you'll receive static help content when available.
Launch Q help
Subnets (1/9) Q help
Last updated 8 minutes ago
Actions
Create subnet
Find subnets by attribute or tag
Name
Subnet ID
State
VPC
Block Public Ac...
IPv4 CIDR
subnetpub-2
subnet-0c7c03cbedd6eee23
Available
vpc-00edb4b4ee5eccdc3 | Myvpc-01
Off
10.0.0.64/26
pvt-sub
subnet-0623b303ac854f674
Available
vpc-00edb4b4ee5eccdc3 | Myvpc-01
Off
10.0.0.16/28
-
subnet-0b66f59923a687138
Available
vpc-0e6c110d36e2844b3
Off
172.31.0.0/20
-
subnet-01d17250c82dba21e
Available
vpc-0e6c110d36e2844b3
Off
172.31.80.0/20



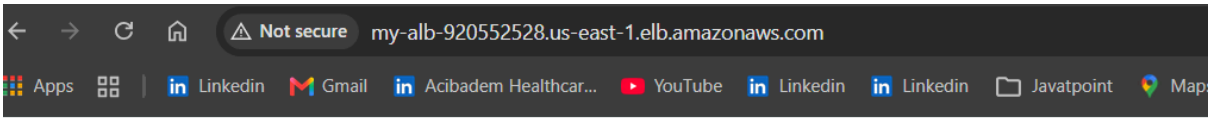
Welcome to Private EC2 Server

Apache Web Server

Apache is successfully running on my Private EC2 instance.

Instance IP: 10.0.0.23

Welcome to Private EC2-2



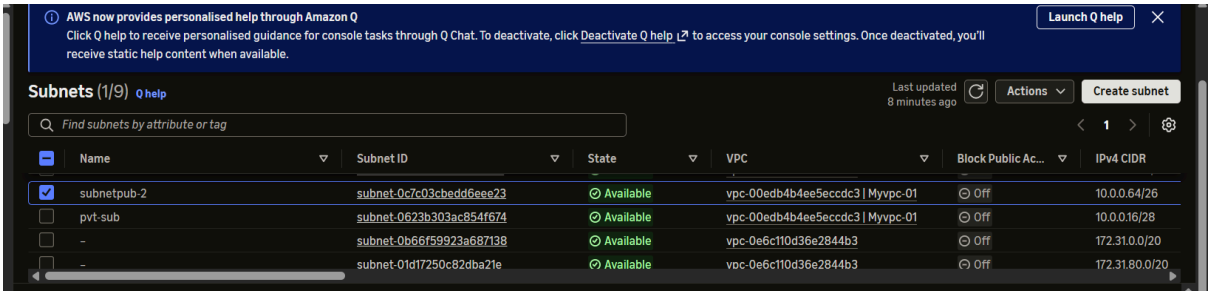
Welcome to Public EC2 Server

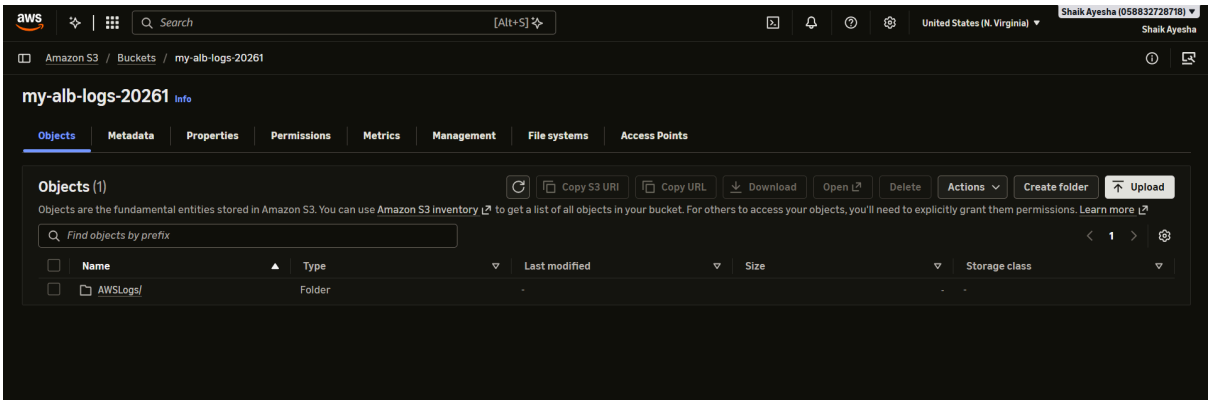
Apache is running successfully on Public EC2.

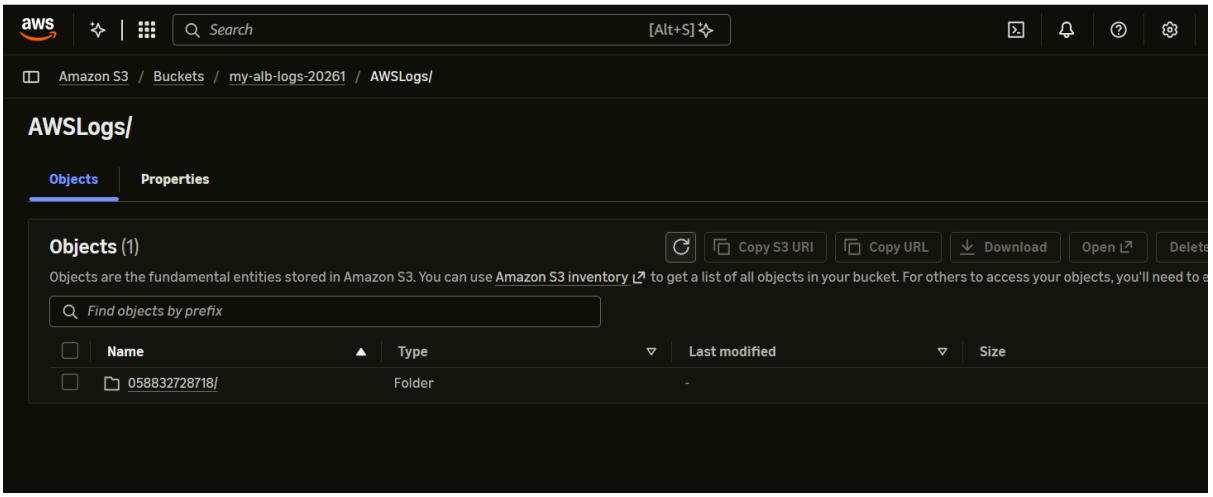
1. I opened the AWS EC2 Console.
2. I selected Load Balancers under Load Balancing.
3. I clicked Create Load Balancer.
4. I selected Application Load Balancer.
5. I entered the Load Balancer name.
6. I selected Internet-facing.
7. I selected IPv4 as the IP address type.
8. I selected my VPC.
9. I selected two public subnets in different Availability Zones.
10. I selected the Security Group that allows HTTP traffic on port 80.
11. I created a new Target Group.
12. I selected Instances as the target type.
13. I entered the Target Group name.
14. I selected HTTP as the protocol and port 80.
15. I selected my VPC.
16. I configured the health check path as /.
17. I selected both EC2 instances.
18. I clicked Include as pending below.

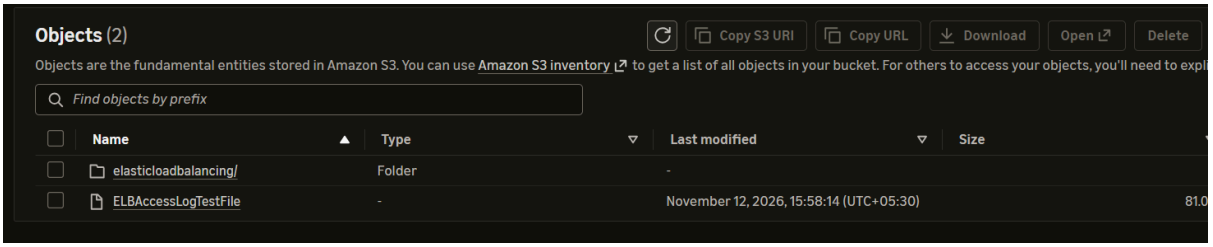
19. I clicked Create Target Group.
20. I returned to the Load Balancer configuration.
21. I selected the newly created Target Group as the default listener action.
22. I clicked Create Load Balancer.
23. I waited until the Load Balancer status changed to Active.
24. I checked both EC2 instances under the Target Group's Targets tab.
25. I verified that both targets showed Healthy.
26. I copied the Load Balancer DNS name.
27. I opened the DNS name in my browser and verified that the application was accessible.

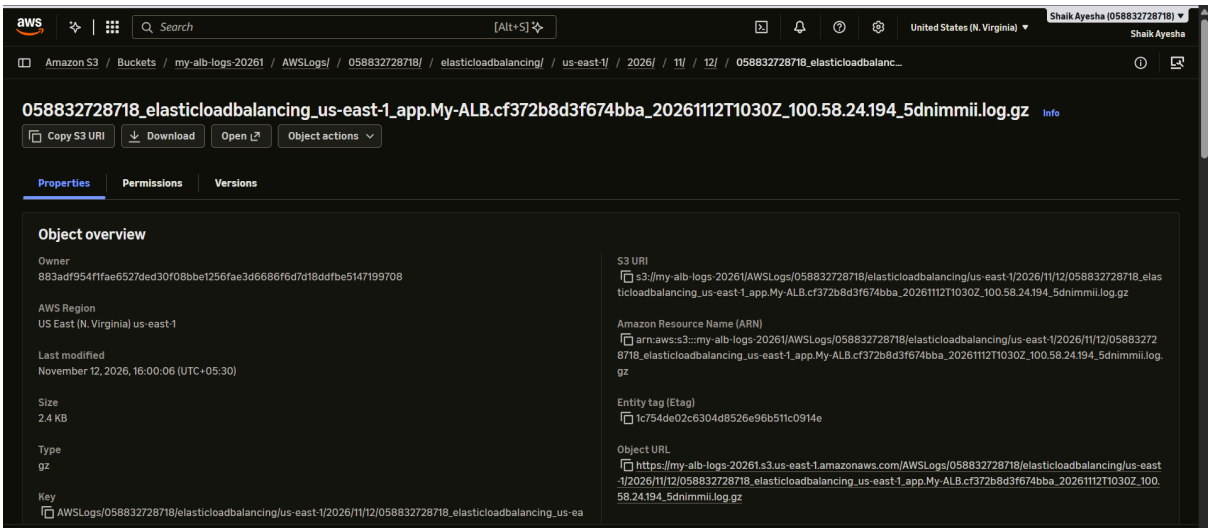
9. Store application load balancer logs in S3.


AWS now provides personalised help through Amazon Q
Click Q help to receive personalised guidance for console tasks through Q Chat. To deactivate, click Deactivate Q help to access your console settings. Once deactivated, you'll receive static help content when available.
Launch Q help
Subnets (1/9) Q help
Last updated 8 minutes ago
Actions
Create subnet
Find subnets by attribute or tag
Name
Subnet ID
State
VPC
Block Public Ac...
IPv4 CIDR
subnetpub-2
subnet-0c7c03cbedd6eee23
Available
vpc-00edb4b4ee5eccdc3 | Myvpc-01
Off
10.0.0.64/26
pvt-sub
subnet-0623b303ac854f674
Available
vpc-00edb4b4ee5eccdc3 | Myvpc-01
Off
10.0.0.16/28
subnet-0b66f59923a687138
Available
vpc-0e6c110d36e2844b3
Off
172.31.0.0/20
subnet-01d17250c82dba21e
Available
vpc-0e6c110d36e2844b3
Off
172.31.80.0/20


Search
[Alt+S]
United States (N. Virginia)
Shaik Ayesha (058832728718)
Shaik Ayesha
Amazon S3 / Buckets / my-alb-logs-20261
my-alb-logs-20261 Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Access Points
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
AWSLogs/
Folder


Search
[Alt+S]
Amazon S3 / Buckets / my-alb-logs-20261 / AWSLogs/
AWSLogs/
Objects
Properties
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to
Find objects by prefix
Name
Type
Last modified
Size
058832728718/
Folder

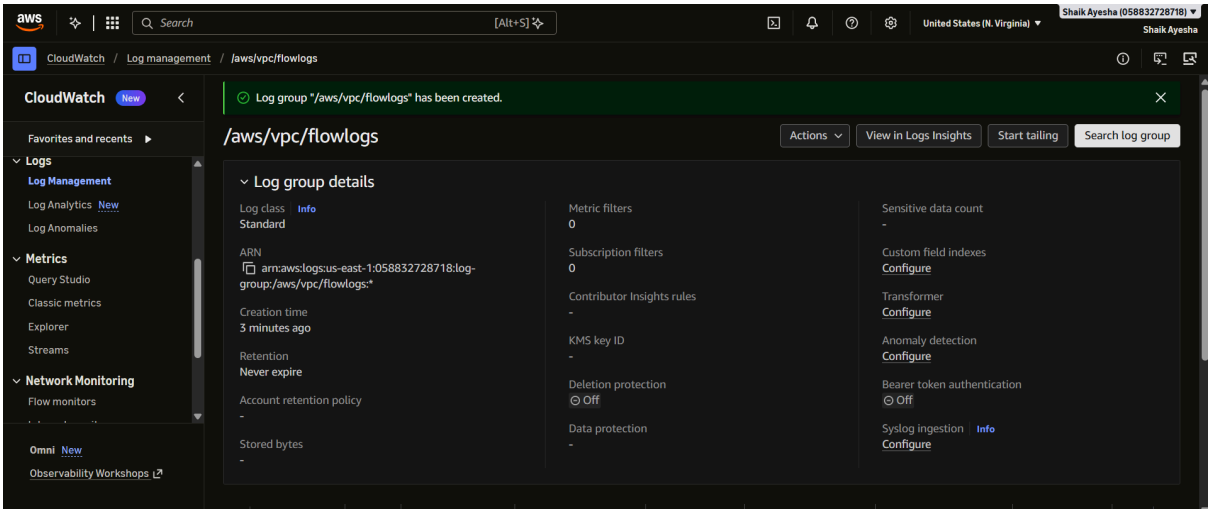


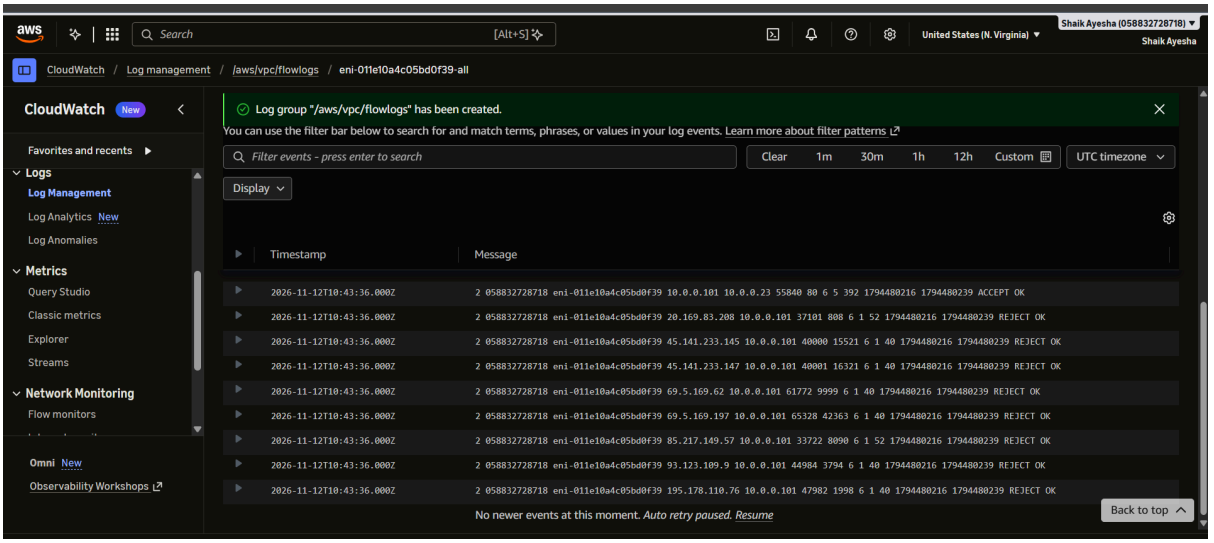


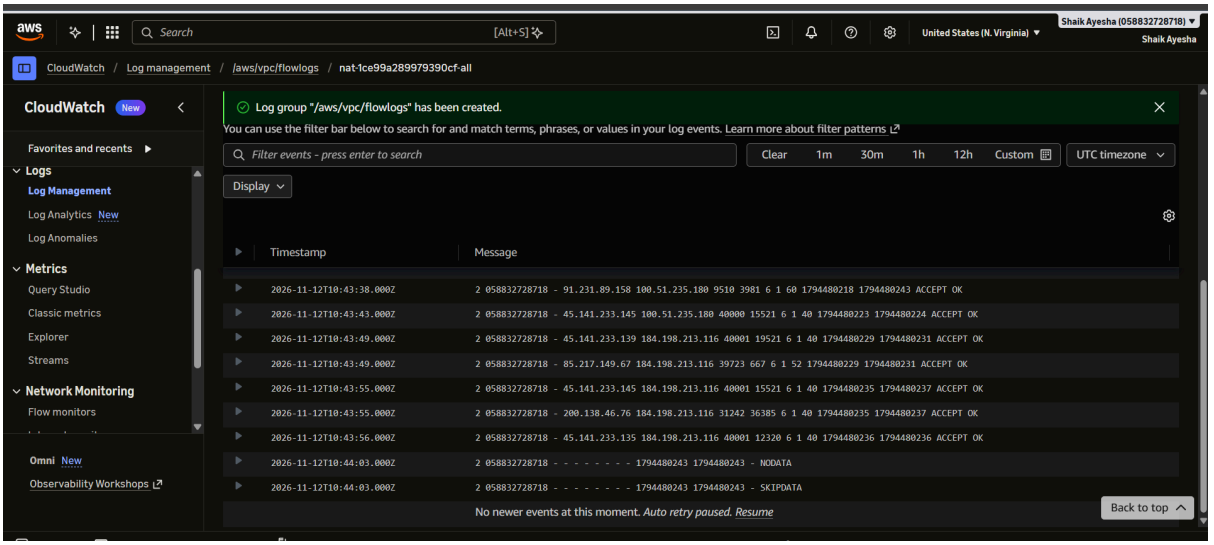
1. I open the AWS Management Console.
2. I search for S3.
3. I click Create bucket.
4. I enter a unique bucket name.
5. I select the same AWS Region as my ALB.
6. I keep Block all public access enabled.
7. I click Create bucket.
8. I open my S3 bucket.
9. I navigate to Permissions → Bucket policy → Edit.
10. I add the ELB log delivery bucket policy with my bucket name and AWS account ID.
11. I click Save changes.
12. I open the EC2 Console → Load Balancers.
13. I select my Application Load Balancer.
14. I open the Attributes tab.
15. I click Edit.
16. I locate Monitoring → Access logs.
17. I enable Access logs.
18. I select my S3 bucket.

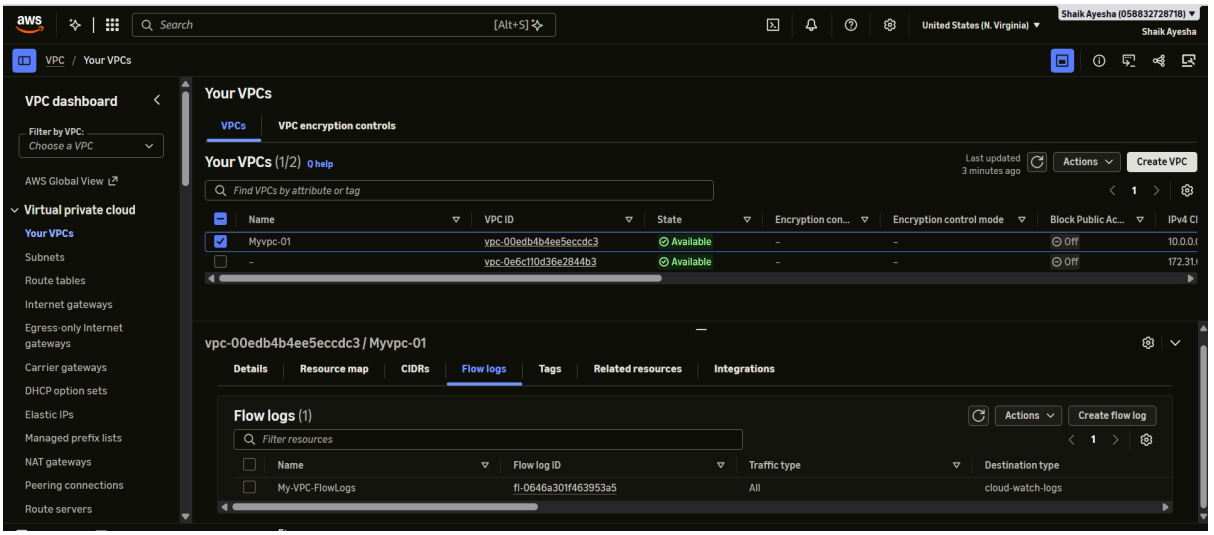
19. I enter my S3 bucket name.
20. I click Save changes.
21. I copy my ALB DNS name.
22. I open the ALB DNS name in a browser to generate traffic.
23. I wait for log delivery.
24. I open my S3 bucket.
25. I navigate to AWSLogs → Account ID → elasticloadbalancing → Region → Year → Month → Date.
26. I verify that .log.gz files are available.

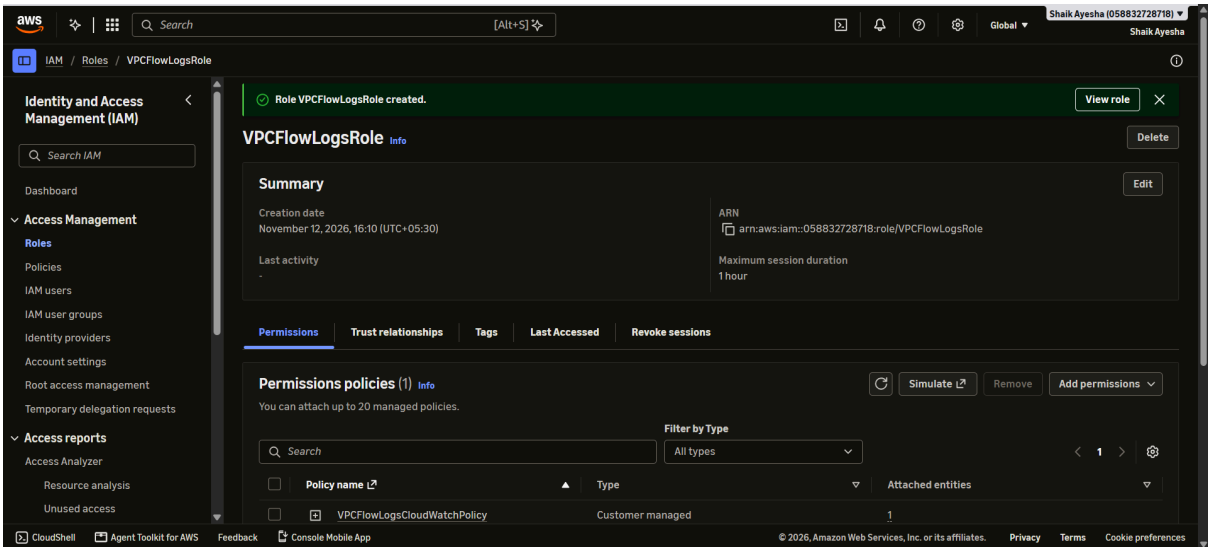
10. Store the VPC flow logs in a CloudWatch log group





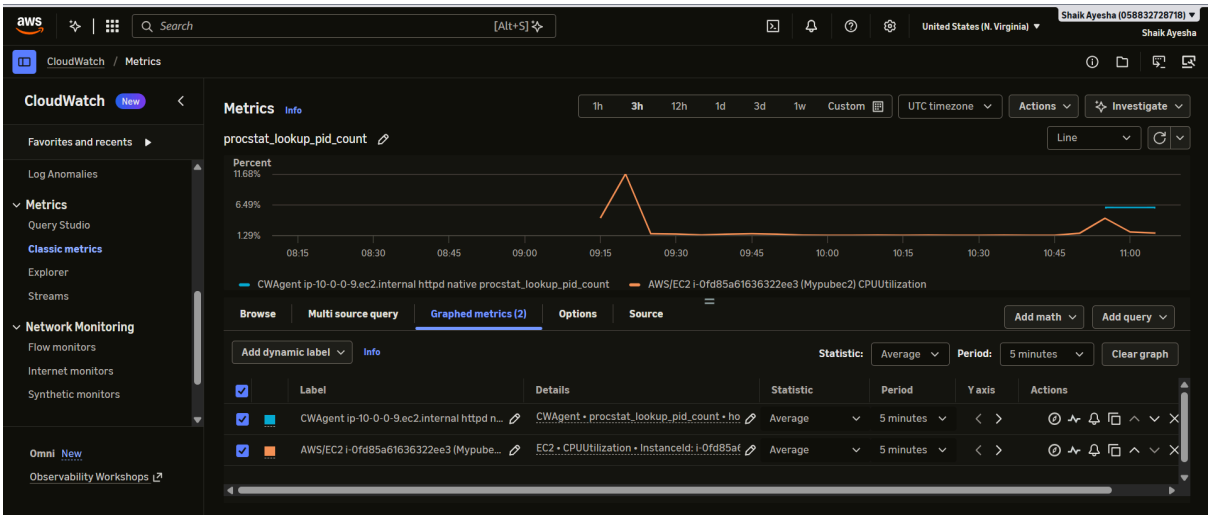

CloudWatch / Log management / /aws/vpc/flowlogs / nat-1ce99a289979390cf-all
Log group "/aws/vpc/flowlogs" has been created.
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Timestamp
Message
2026-11-12T10:43:38.000Z 2 058832728718 - 91.231.89.158 100.51.235.180 9510 3981 6 1 60 1794480218 1794480243 ACCEPT OK
2026-11-12T10:43:43.000Z 2 058832728718 - 45.141.233.145 100.51.235.180 40000 15521 6 1 40 1794480223 1794480224 ACCEPT OK
2026-11-12T10:43:49.000Z 2 058832728718 - 45.141.233.139 184.198.213.116 40001 19521 6 1 40 1794480229 1794480231 ACCEPT OK
2026-11-12T10:43:49.000Z 2 058832728718 - 85.217.149.67 184.198.213.116 39723 667 6 1 52 1794480229 1794480231 ACCEPT OK
2026-11-12T10:43:55.000Z 2 058832728718 - 45.141.233.145 184.198.213.116 40001 15521 6 1 40 1794480235 1794480237 ACCEPT OK
2026-11-12T10:43:55.000Z 2 058832728718 - 200.138.46.76 184.198.213.116 31242 36385 6 1 40 1794480235 1794480237 ACCEPT OK
2026-11-12T10:43:56.000Z 2 058832728718 - 45.141.233.135 184.198.213.116 40001 12320 6 1 40 1794480236 1794480236 ACCEPT OK
2026-11-12T10:44:03.000Z 2 058832728718 - - - - - - - - 1794480243 1794480243 - NODATA
2026-11-12T10:44:03.000Z 2 058832728718 - - - - - - - - 1794480243 1794480243 - SKIPDATA
No newer events at this moment. Auto retry paused. Resume
Back to top

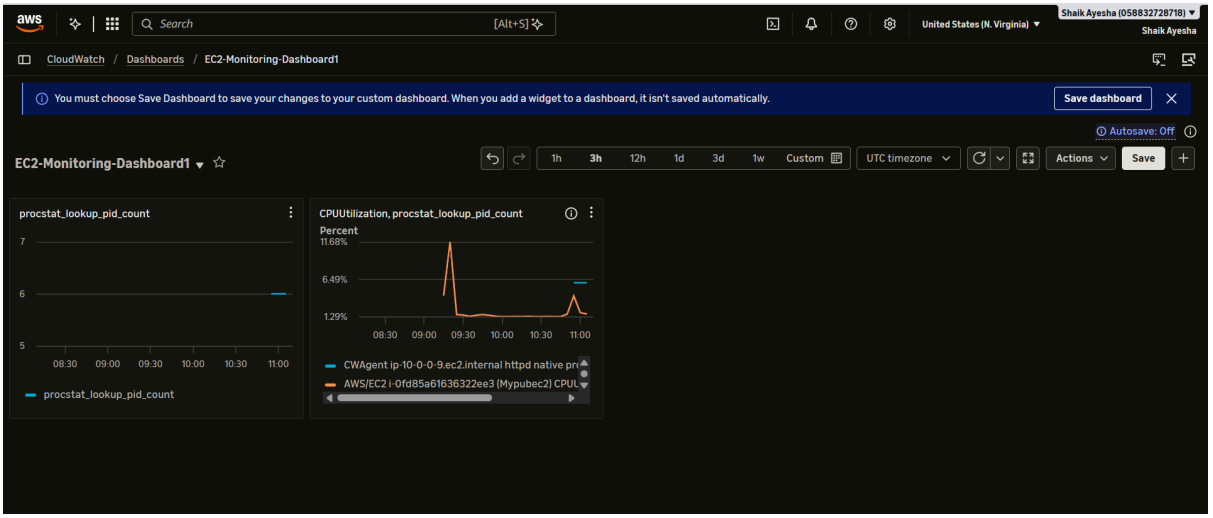

VPC / Your VPCs
Your VPCs
VPC dashboard
Your VPCs (1/2)
Myvpc-01
vpc-00edb4b4ee5eccdc3
vpc-0e6c110d36e2844b3
vpc-00edb4b4ee5eccdc3 / Myvpc-01
Flow logs (1)
My-VPC-FlowLogs
fl-0646a301f463953a5


IAM / Roles / VPCFlowLogsRole
Role VPCFlowLogsRole created.
VPCFlowLogsRole
Summary
Creation date
November 12, 2026, 16:10 (UTC+05:30)
ARN
arn:aws:iam::058832728718:role/VPCFlowLogsRole
Permissions policies (1)
VPCFlowLogsCloudWatchPolicy
Customer managed

1. I open the AWS Management Console.
2. I navigate to VPC → Your VPCs.
3. I select my VPC.
4. I open the Flow Logs tab.
5. I click Create flow log.
6. I enter a Flow Log name.
7. I select Filter: All.
8. I select Destination: Send to CloudWatch Logs.
9. I create a new CloudWatch Log Group or select an existing one.
10. I create an IAM role with the required permissions to publish flow logs to CloudWatch.
11. I select the IAM role.
12. I select the log record format as AWS default format.
13. I select the maximum aggregation interval (1 minute or 10 minutes).
14. I click Create flow log.
15. I navigate to CloudWatch → Logs → Log groups.
16. I open my CloudWatch Log Group.
17. I verify that VPC Flow Logs are being delivered.

11. Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.





Part 1: CPU utilization

1. I will log in to AWS Console.
2. I will open EC2 → Instances.
3. I will select my running EC2 instance.
4. I will note down the Instance ID.
5. I will open CloudWatch → Metrics → All metrics.
6. I will select EC2 → Per-Instance Metrics.
7. I will select my Instance ID.
8. I will select CPUUtilization.
9. I will click Actions → Add to dashboard.
10. I will create a dashboard named EC2-Monitoring-Dashboard1.
11. I will add the CPU metric and save the dashboard.

Part 2: Apache service monitoring

1. I will connect to my EC2 instance using SSH.
2. I will install Apache.

```
None
        sudo dnf install httpd -y
```

3. I will start Apache.

```
None
        sudo systemctl start httpd
```

4. I will enable Apache.

```
None
        sudo systemctl enable httpd
```

5. I will create an IAM role with CloudWatchAgentServerPolicy.
6. I will attach the IAM role to my EC2 instance.
7. I will install CloudWatch Agent.

```
None
        sudo dnf install amazon-cloudwatch-agent -y
```

8. I will create the configuration file.

```
None
        sudo vi
        /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
```

9. I will add the Apache procstat configuration.
10. I will save the file.
11. I will start CloudWatch Agent.

```
None
        sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl
        -a fetch-config -m ec2 -c
        file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json
        -s
```

12. I will verify the agent status.

```
None
        sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl
        -a status
```

13. I will open CloudWatch → Metrics → All metrics.
14. I will select CWAgent → procstat_lookup_pid_count.
15. I will select the httpd metric.

16. I will add the metric to EC2-Monitoring-Dashboard1.
17. I will save the dashboard.

Part 3: Verify monitoring

1. I will open CloudWatch → Dashboards.
2. I will open EC2-Monitoring-Dashboard1.
3. I will verify CPU utilization and Apache process count graphs.
4. I will stop Apache to test monitoring.

```
None
        sudo systemctl stop httpd
```

5. I will verify the Apache process count.
6. I will start Apache again.

```
None
        sudo systemctl start httpd
```

7. I will verify that the Apache process count increases.

12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance

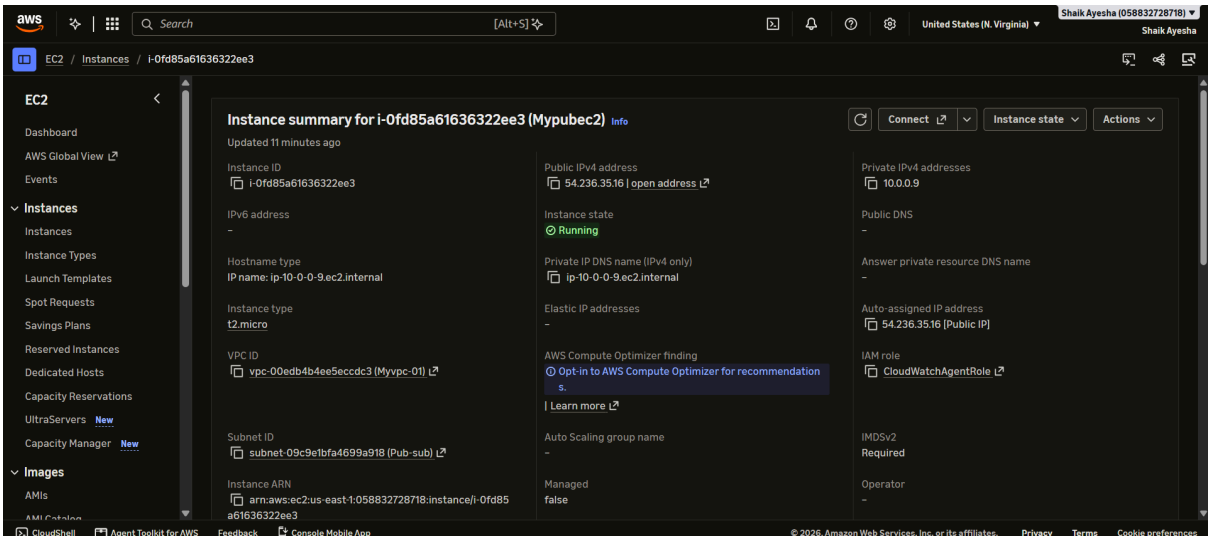
```
ec2-user@ip-10-0-0-9:~
[ec2-user@ip-10-0-0-9 ~]$ sudo dnf install stress-ng -y
Last metadata expiration check: 2:39:58 ago on Thu Nov 12 09:22:00 2026.
Dependencies resolved.
================================================================================
 Package              Architecture     Version                    Repository          Size
================================================================================
Installing:
 stress-ng            x86_64           0.15.05-1.amzn2023         amazonlinux        2.3 M
Installing dependencies:
 Judy                 x86_64           1.0.5-25.amzn2023.0.3      amazonlinux        153 k
 libbsd               x86_64           0.10.0-7.amzn2023.0.2      amazonlinux        109 k
 lksctp-tools         x86_64           1.0.18-9.amzn2023.0.3      amazonlinux         92 k

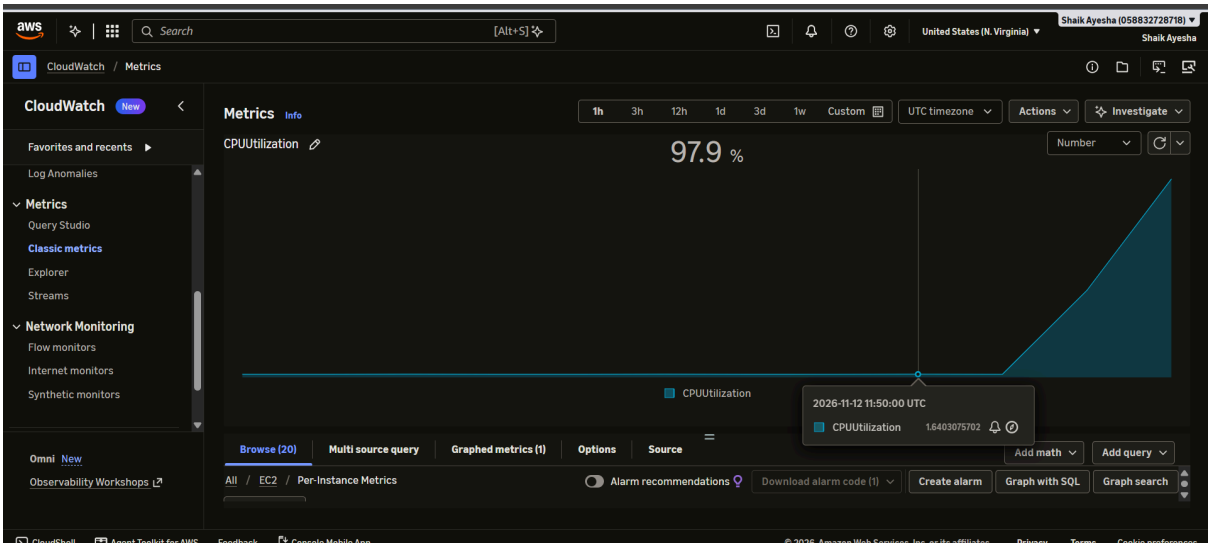
Transaction Summary
================================================================================
Install  4 Packages

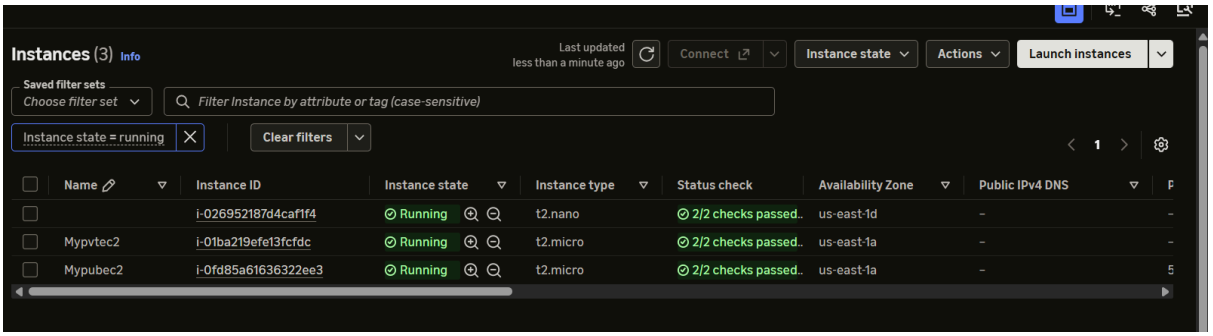
Total download size: 2.7 M
Installed size: 9.7 M
Downloading Packages:
(1/4): Judy-1.0.5-25.amzn2023.0.3.x86_64.rpm                 8.0 MB/s | 153 kB     00:00
(2/4): libbsd-0.10.0-7.amzn2023.0.2.x86_64.rpm               4.7 MB/s | 109 kB     00:00
(3/4): lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64.rpm         3.3 MB/s |  92 kB     00:00
(4/4): stress-ng-0.15.05-1.amzn2023.x86_64.rpm                39 MB/s | 2.3 MB     00:00
--------------------------------------------------------------------------------
Total                                                         20 MB/s | 2.7 MB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Installing       : lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64              1/4
  Installing       : libbsd-0.10.0-7.amzn2023.0.2.x86_64                    2/4
  Installing       : Judy-1.0.5-25.amzn2023.0.3.x86_64                      3/4
  Installing       : stress-ng-0.15.05-1.amzn2023.x86_64                    4/4
  Running scriptlet: stress-ng-0.15.05-1.amzn2023.x86_64                    4/4
  Verifying        : Judy-1.0.5-25.amzn2023.0.3.x86_64                      1/4
  Verifying        : libbsd-0.10.0-7.amzn2023.0.2.x86_64                    2/4
  Verifying        : lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64              3/4
  Verifying        : stress-ng-0.15.05-1.amzn2023.x86_64                    4/4

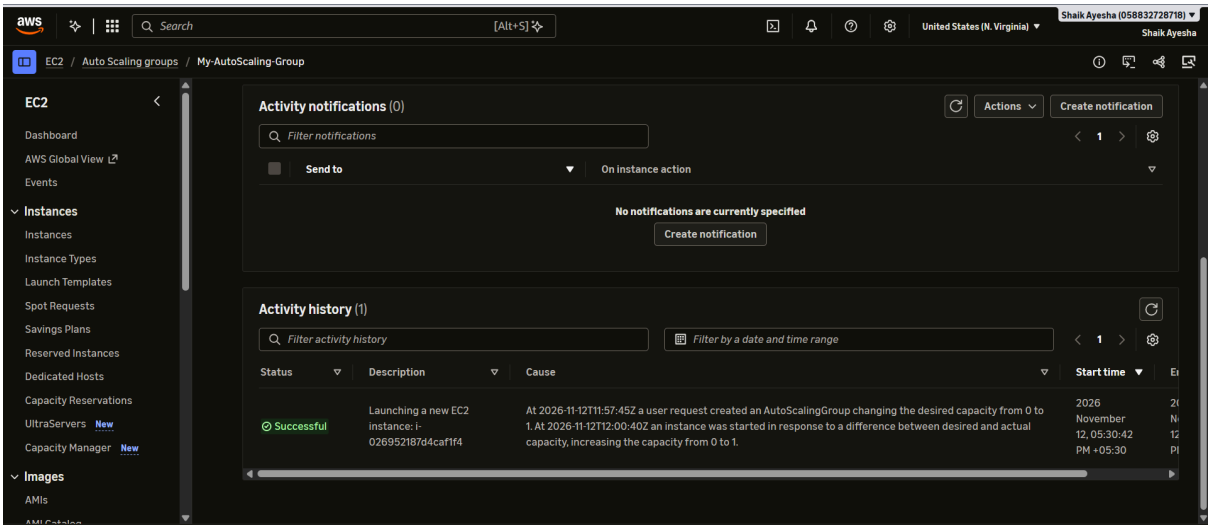
Installed:
  Judy-1.0.5-25.amzn2023.0.3.x86_64                 libbsd-0.10.0-7.amzn2023.0.2.x86_64
  lksctp-tools-1.0.18-9.amzn2023.0.3.x86_64         stress-ng-0.15.05-1.amzn2023.x86_64

Complete!
[ec2-user@ip-10-0-0-9 ~]$ stress-ng --cpu 2 --cpu-load 90 --timeout 10m
stress-ng: info:  [36249] setting to a 600 second (10 mins, 0.00 secs) run per stressor
stress-ng: info:  [36249] dispatching hogs: 2 cpu
```









Step 1: Create Launch Template

1. I opened the AWS EC2 console.
2. I selected Launch Templates.
3. I clicked Create launch template.
4. I entered the launch template name.
5. I selected the Amazon Linux 2023 AMI.

6. I selected the instance type.
7. I selected the key pair and security group.
8. I clicked Create launch template.

Step 2: Create Auto Scaling Group

1. I opened EC2 → Auto Scaling Groups.
2. I clicked Create Auto Scaling group.
3. I entered the Auto Scaling Group name.
4. I selected my launch template.
5. I selected my VPC and subnets.
6. I configured the load balancer attachment if required.
7. I clicked Next.

Step 3: Configure Scaling Policy

1. I configured the following capacity settings:
    - Desired capacity: 1
    - Minimum capacity: 1
    - Maximum capacity: 3
2. I selected Target Tracking Scaling Policy.
3. I entered the scaling policy name.
4. I selected Average CPU Utilization.
5. I configured the target value as 70%.
6. I configured instance warmup as 300 seconds.
7. I created the Auto Scaling Group.

Step 4: Generate CPU Load

1. I connected to my EC2 instance using SSH.
2. I installed the stress-ng utility.

```
None
sudo dnf install stress-ng -y
```

3. I generated CPU load using the following command:

```
None
stress-ng --cpu 2 --cpu-load 90 --timeout 15m
```

Step 5: Monitor CPU Utilization

1. I opened CloudWatch.
2. I selected Metrics → EC2 → Per-Instance Metrics.
3. I selected CPUUtilization.
4. I monitored CPU utilization.
5. I verified that CPU utilization increased above 70%.

Step 6: Verify Auto Scaling

1. I opened EC2 → Auto Scaling Groups.
2. I selected my Auto Scaling Group.
3. I opened the Activity tab.
4. I verified the scaling activity.
5. I opened Instance management.
6. I verified that a new EC2 instance was launched automatically.
7. I confirmed that the instance count increased from 1 to 2.